TABLE OF CONTENTS

1. INTRODUCTION

1.1 Purpose and Scope

The purpose of the Smart Study Planner final project is to deliver a mobile study planning system that helps a student manage courses, tasks, exams, focus time, and daily progress in one place. The application is not only a static to-do list. It records what the student finished, what was postponed, and which course still needs attention.

In the final version, the word Subject in the backend is presented to the user as Course. This was kept because the earlier database model already used Subject, while the student experience is more natural when the screen says Calculus, Physics, or any university course name. The scope covers a React Native mobile application, an ASP.NET Core backend running on version 8, and a Python/FastAPI AI engine.

The project now includes onboarding, course creation with optional midterm and final exam dates, calendar task creation, course detail progress, focus timer sessions, simple analytics, and an AI improvement option. The first daily plan can be generated by the system itself using rules.

1.2 Problem Statement

Students usually do not fail only because they do not have a calendar. They fail because the calendar does not know how hard a course is, which tasks were finished, how much time the student has today, and what happens when the student moves between courses.

1. Static scheduling failure: a fixed plan does not react when a task moves, a session is interrupted, or a course becomes harder than expected.
2. Weak course visibility: many students can see a list of tasks, but they cannot quickly understand the state of each course alone.
3. Task entry confusion: if adding tasks is heavy, the student stops using the application before the planner becomes useful.

1.3 Solution Statement

The final solution organizes the student workflow around courses, calendar days, focus sessions, and small progress signals. A course can have a midterm date, a final date, both dates, or no exam date at all. This is important because not every course in a university semester is evaluated in the same way.

Tasks can be created from the calendar or the course screens. Completed tasks are still visible as done, and the application uses progress messages so that the student sees what was gained instead of only seeing what remains.

1.4 Contribution

This final project contributes a practical student planner that combines manual control with automatic support. It tries to keep the application simple for daily use, while still keeping enough data for backend analytics and AI optimization.

- A course-first planning model where each course has priority, difficulty, optional midterm date, optional final date, and task progress.
- A calendar workflow where tasks are created and viewed by date, including completed tasks with a clear done state.
- A focus workflow that records actual study time, supports moving between courses, and sends focus ratings to the backend.
- A rule-based schedule generator that works without AI, with AI optimization available as an additional option.

1.5 Glossary

- AI Engine: The Python/FastAPI service that computes analysis values, difficulty factors, and optional AI schedule output.
- Available Slot: A saved time range that tells the planner when the student can study.
- Course / Subject: The academic unit such as Calculus or Physics. The UI says Course, while the backend entity name remains Subject.
- Focus Session: A timed study or break session connected to a course and optionally to a task.
- Rule-Based Plan: The non-AI scheduling method that orders tasks using date, priority, difficulty, and available time.
- Study Pulse: The analytics view that summarizes streak, done tasks, open tasks, focus, and weekly progress.

The final report therefore continues the same project described in the midterm report, but the focus is now on the completed implementation and how the screens, API, database, and AI service work together.

2. LITERATURE REVIEW

The Smart Study Planner is based on the same academic direction that was introduced in the midterm phase: planning fallacy, cognitive fatigue, educational data mining, and adaptive education. In the final phase, this literature is applied in a more concrete product flow.

2.1 The Planning Fallacy and Static Scheduling Systems

The planning fallacy explains why people normally underestimate how long future work will take. For a student, this appears when a chapter is expected to take one hour but actually needs two or three sessions. A planner that only stores the first estimate will repeat the same mistake. The final system therefore stores estimated and actual minutes, and the course difficulty value can influence future planning.

2.2 Cognitive Fatigue and Academic Burnout Prediction

Academic burnout and cognitive fatigue are not only emotional states, they also appear in behavior. Long continuous sessions, low focus rating, repeated postponing, and weak completion rhythm can show that the student is overloaded. The application does not diagnose the student, but it uses these signals to make the plan more careful and to show the student small warnings and progress information.

2.3 Machine Learning in Educational Data Mining (EDM)

Educational Data Mining uses student data to find useful patterns for learning systems. In this project, the final AI engine uses cold-start heuristics first, then can use personalized models when enough data is collected. This is better than forcing all students into one general model from the first day.

2.4 Large Language Models (LLMs) in Adaptive Education

Large Language Models can help transform structured data into a readable and adaptive plan. Still, an LLM alone is not enough, because it may not know the real constraints of the application. For that reason, the backend prepares course, task, slot, focus, and behavioral values first, and the AI engine treats the LLM as a controlled scheduling assistant, not as an open chat feature.

The final product also allows the daily plan to be created without AI. This design choice is important for reliability and user trust. The AI is useful, but the student must still be able to generate a plan when the AI service is unavailable or when he prefers a faster simple plan.

2.5 Summary and Identified Gap in the Literature

The gap remains clear: normal calendars do not understand study difficulty, and many AI tools are too generic for daily student planning. The Smart Study Planner fills this gap by combining a normal usable mobile workflow with backend data and optional AI improvement. The final application tries to stay simple because if the workflow is complicated, the student will not continue entering data.

This was also a design lesson. The planner should not feel like a task camp. It should feel like a study companion that gently shows what is done, what remains, and what deserves attention today.

The literature therefore supports three final decisions: course difficulty should be visible, analytics should stay simple, and artificial intelligence should improve the plan without removing the student's control.

These decisions were important in the final design because the student needs an application that is useful from the first day. The system must help even before there is enough historical data for a trained model. For this reason, the rule-based planning path and the course difficulty input are not temporary parts; they are part of the real final workflow.

The reviewed studies also show that motivation is connected with feedback. A student who sees only missed tasks may stop planning, while a student who sees completed work, a small streak, and course progress can understand the next step more calmly. This idea was reflected in the final Study Pulse and course detail screens.

3. SOFTWARE REQUIREMENTS SPECIFICATION

3.1 Introduction

This section defines the final requirements of the Smart Study Planner after implementation. The system is a mobile-first application supported by a backend API and an AI microservice. The requirements are written according to the same organization used in the midterm report, with the final implementation details updated to match the actual source code.

3.2 The Overall Description

The final product has three main parts: mobile app, backend API, and AI service. The mobile app is used by students. The backend stores users, courses, tasks, focus sessions, available slots, schedules, and analytics data. The AI service receives structured planning data and returns optimized schedules and analysis results when the AI option is selected.

3.2.1 Product Perspective

Smart Study Planner is designed as an independent academic planning product. It can run locally during development and also runs on a temporary HTTPS deployment. The deployment is separated from other server projects by using an independent temporary directory, so removing that directory later does not affect the existing unrelated website on the same machine.

3.2.1.1 System Interfaces

The system interfaces are HTTP JSON APIs. The React Native frontend communicates with the backend through authenticated API calls. The backend communicates with the AI engine through an internal optimize schedule request. Health checks are used to verify backend, database, and AI service readiness.

3.2.1.2 Interfaces (User Interfaces)

The final user interface contains sign in and sign up, onboarding, home dashboard, calendar, courses, course detail, focus, Study Pulse analytics, and profile/settings screens. The main bottom navigation keeps the repeated student workflow close: Home, Calendar, Courses, Focus, and Profile.

- Authentication screen uses email and password. The final report screenshots use a team-like demo email for a realistic demo.
- Onboarding asks for name, deadline, first course, difficulty, priority, optional midterm/final, first task, and study slots.
- Calendar allows the student to choose a date and create tasks for that date.
- Course detail shows progress, open tasks, done tasks, exam dates when they exist, and course-level attention details.

3.2.1.3 Hardware Interfaces

The mobile application runs on a smartphone or Android emulator. No special hardware is required beyond normal phone storage, screen, and network access. The backend and AI engine run on a server or local development machine with enough memory for the backend runtime, Python, and the database process.

3.2.1.4 Software Interfaces

The frontend is implemented with React Native and Expo. The backend is implemented with ASP.NET Core version 8, Entity Framework Core, Identity password hashing, JWT authentication, and SQLite for the current temporary deployment database. The AI engine is implemented with FastAPI and Python ML modules.

3.2.1.5 Communications Interfaces

All application communication is performed through HTTPS in deployment and HTTP in local development. The backend exposes REST endpoints for authentication, users, courses, tasks, focus sessions, available slots, schedule, analytics, and behavioral logs. The AI service exposes an optimize schedule endpoint.

3.2.1.6 Memory Constraints

The mobile app stores only the data needed for the current logged-in user and relies on the backend as the source of truth. The backend stores schedule payloads and analysis values, while the AI service loads only the models and request data needed for a schedule call. The final product does not require heavy device-side machine learning.

3.2.1.7 Operations

Normal operation starts with authentication. A new user completes onboarding by adding the first course, task, and study availability. After that, the user can create more courses and tasks, complete tasks, start focus sessions, generate a daily plan, improve the plan with AI, and monitor progress from Study Pulse.

3.2.1.8 Site Adaptation Requirements

The temporary deployment uses independent service configuration, a separated project directory, HTTPS routing through sslip.io style domains, and GitHub Actions for updates. This lets the project be tested for the final demo without mixing it with the original server website.

3.3 Product Functions

1. Register and login using email and password with backend validation and token-based sessions.
2. Complete onboarding without a skip path, so the planner starts with enough useful data.
3. Create, update, and view courses, including priority, difficulty, midterm date, and final date.
4. Allow a course to have no midterm, no final, or both dates depending on the real class structure.
5. Create tasks from calendar and course flows, then mark tasks as done while keeping them visible.
6. Generate a daily plan without AI and optionally request AI improvement.
7. Start, pause, and finish focus sessions while recording actual study time and focus rating.
8. Display simple analytics: streak, done tasks, open tasks, study hours, focus score, and course momentum.

3.4 User Characteristics

The main user is a university student who has several courses and needs help deciding what to study next. The user may not want to enter every chapter in detail, so the application supports simple task creation and leaves detailed AI improvement as an optional second step.

3.5 Constraints

- The application must keep the daily workflow simple enough for repeated use.
- The planner must work without depending on AI for every schedule generation.
- Course exam dates must be optional because not all courses have both midterm and final exams.
- The backend must protect user data with authentication and user-scoped database queries.
- The temporary server deployment must not interfere with the existing website on the same server.

3.6 Assumptions and Dependencies

It is assumed that the student has an active network connection when syncing data. It is also assumed that the student will enter at least one course and one task during onboarding. AI optimization depends on the AI service and the configured Gemini provider, but the basic schedule is not blocked by this.

3.7 Apportioning of Requirements

Requirements were divided into implemented core features and future enhancements. The implemented core features are authentication, onboarding, courses, calendar tasks, focus sessions, schedule generation, AI improvement, analytics, deployment, and CI/CD. Future work is limited to polish items and advanced integrations, not required for the final demo.

3.8 Specific Requirements

3.8.1 Functions

Authentication	Register, login, refresh token, current user, change password, forgot password stub.
Courses	List, create, update, delete, optional midterm/final, priority, difficulty.
Tasks	Create, list by filter, update, complete, attach actual minutes and status.
Focus	Start and complete sessions, save course, task, mode, elapsed time, focus rating.
Schedule	Generate plan by rule-based logic, call AI when useAi is true, save latest schedule.
Analytics	Return streak, focus score, done/open counts, today hours, badges, weekly rhythm.

3.8.2 Performance Requirements

The main mobile screens should load quickly from cached React state and API refreshes. Backend endpoints should return normal course, task, and analytics data within a short interactive delay. AI calls may take longer, so the UI treats AI as an optional improvement and keeps the normal schedule available.

3.8.3 Logical Database Requirements

The logical database must store users, courses, study tasks, focus sessions, available slots, AI schedules, and behavioral logs. Every student-owned entity must include the user relation so that one user's courses and tasks cannot appear for another user.

User	id, name, email, password hash, deadline, onboarding status, refresh tokens.
Subject	id, user id, name, difficulty, priority, exam date, midterm date, final date.
StudyTask	id, user id, subject id, title, deadline, priority, estimate, actual, status.
FocusSession	id, user id, subject id, task id, mode, started, ended, rating.
AvailableSlot	id, user id, day/date, start time, end time.

3.8.4 Design Constraints

The design uses a mobile-first layout with large touch targets, direct navigation, and calm analytics. The application should not look like a punishment system for tasks. It should show progress, unfinished work, and next steps in a way that feels motivating for a student.

3.8.4.1 Standards Compliance

- The backend follows REST-style JSON endpoints with HTTP status codes.
- Authentication follows JWT bearer token usage.
- The frontend follows React Native component structure and Expo development workflow.
- CI/CD uses GitHub Actions with repeatable build and deploy steps.

3.8.5 Software System Attributes

3.8.5.1 Reliability

The planner must stay usable when AI is not selected or when the AI service fails. This is why the backend includes a rule-based schedule path and a fallback response for AI calls.

3.8.5.2 Availability

The deployed services are kept independent inside the temporary server directory and checked through health endpoints. The mobile app can connect to the hosted backend during the demo through HTTPS.

Software Attribute Summary

The software attributes were reviewed as a group before writing the final report. Reliability, availability, security, maintainability, and portability are connected because the planner must run as a complete system, not only as separate screens.

Requirement Traceability Notes

The backend separates controllers, application services, infrastructure, domain entities, DTOs, and validators. The frontend separates screens, contexts, components, theme, and API services. This makes future changes easier because a screen improvement does not require rewriting the backend model.

Deployment and Environment Notes

The mobile app can run in Expo development, web preview, and Android emulator. The backend and AI services can run locally or on the temporary Linux server. Configuration values are kept outside source code when they are environment-specific.

Final Requirement Organization Check

The requirements are organized around the student's journey: login, onboarding, courses, calendar tasks, daily planning, focus execution, analytics, and deployment verification. This order matches how the final demo is expected to be shown.

Final requirement traceability was checked against the frontend screens and backend API modules. The following behavior is visible in the implemented application: courses appear with exam metadata, calendar tasks can be created by date, Study Pulse stays simple, and the user can still request AI when needed.

REQ-01	Email/password authentication is implemented in the Auth API and mobile login screen.
REQ-02	Course dates are optional and are displayed only when entered by the student.
REQ-03	Calendar creates and displays tasks for the selected day.
REQ-04	Completed tasks update status and analytics counts.
REQ-05	Focus session records real time and rating.
REQ-06	Schedule can be generated with or without AI.

3.8.5.3 Security

In the final implementation, security is handled mainly in the backend. The frontend does not decide which data belongs to a user. It sends the authenticated request, then the backend checks the token and loads only records connected to that user. This is important for courses and tasks because two students may have a course with the same name, but each student must see only his own data.

- Passwords are validated before registration and stored as hashed values.
- Access tokens are used for authenticated requests and refresh tokens support longer sessions.
- Course, task, focus, and analytics endpoints are scoped to the current authenticated user.
- Deployment secrets and server credentials are kept outside the final report and outside public UI text.

3.8.5.4 Maintainability

Maintainability was improved by keeping the frontend, backend, and AI service separated. This allows a UI change, such as improving the calendar task creation flow, without changing the AI model code. It also allows backend validation changes without rebuilding the whole mobile interface.

3.8.5.5 Portability

The final project can be moved between local development and temporary hosting. The mobile app can run through Expo, web preview, and Android emulator testing. The backend and AI services use normal server processes, while configuration values decide which URL and database are used in each environment.

3.8.6 Organizing the Specific Requirements

The final requirements are organized by the student's real path through the product. First the user logs in, then completes onboarding, then adds courses and tasks, then studies through focus sessions, and then checks progress. This order keeps the final report aligned with the demo flow.

Student setup	Register, login, onboarding, first course, first task, available slots.
Daily planning	Calendar date, task creation, simple plan, optional AI.
Study execution	Focus timer, course switching, task completion, actual minutes, rating.
Progress review	Course detail, done/open counts, streak, weekly rhythm, badges.

Final SRS Review

The SRS was reviewed after implementation so that it does not describe features that are not present in the code. For example, the final report does not depend on external calendar sync or social login. The implemented authentication is email/password, and the calendar is an internal student planning calendar.

This review also clarified the course model. A course can have only a final, only a midterm, both exam dates, or no exam dates. This behavior matches how real university courses are different from each other, and it prevents the application from forcing fake exam data.

The final SRS therefore represents what the team can demonstrate: a working study planner that helps a student understand today's work, the state of each course, and whether his study rhythm is improving.

4. SYSTEM DESIGN SPECIFICATION

4.1 Introduction

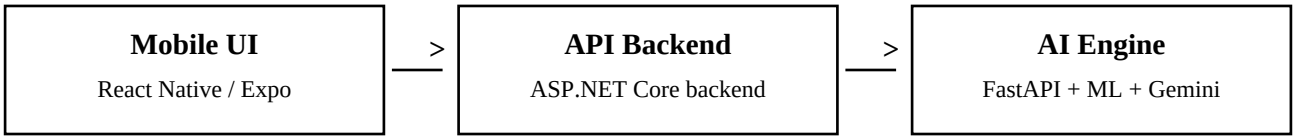
The system design connects a student-facing mobile application with a backend API and an AI microservice. The final design keeps the student workflow simple while preserving enough structure for planning, analytics, and AI optimization. The design also follows the same main structure presented in the midterm report, but the details are updated to the final implementation.

The most important design change in the final phase is that AI is not the only path. The planner can create a daily program by itself using backend rules, then AI can be used for improvement. This makes the application more stable and easier for students to trust.

4.2 SYSTEM ARCHITECTURE

4.2.1 System Software Architecture

The architecture is divided into three layers. The React Native application handles interaction and visual feedback. The ASP.NET Core backend handles authentication, business logic, persistence, and schedule orchestration. The FastAPI AI engine receives structured planning context and returns analysis or an improved plan.



Identity	Email/password, hashed password, JWT access and refresh token flow.
Planning data	Courses, tasks, available slots, focus sessions, behavioral logs.
Scheduling	Rule-based daily schedule first, optional AI improvement as second pass.
Analytics	Streak, completed work, open work, study hours, focus score, course momentum.

The backend is the central authority. It does not allow the mobile app to write arbitrary user state without validation. It checks the current user, verifies that courses and tasks belong to that user, and then saves the requested operation. This also made the final hosted demo more reliable because all important state is saved through the API.

- AuthController handles register, login, refresh, current user, change password, and forgot password placeholder.
- SubjectsController handles course data even though the domain entity name remains Subject.
- TasksController handles task creation, update, completion, and filtered lists.
- FocusSessionsController stores focus sessions and ratings.
- ScheduleController generates rule-based plans or AI-improved plans.
- AnalyticsController returns Study Pulse values used in the mobile screen.

The frontend state is managed through contexts and API hooks. This makes repeated screens easier to keep consistent. For example, completing a task should affect the calendar, courses, dashboard, and analytics. The shared AI/context layer refreshes data after changes so the student does not have to manually reload.

The AI service has a controlled responsibility. It is not responsible for user authentication or database ownership. It receives a planning request from the backend, analyzes the study data, and returns schedule slots and analysis results. This separation prevents the AI code from becoming mixed with user account logic.

For deployment, the final version uses a temporary isolated server directory and CI/CD. The GitHub Actions workflow deploys the frontend, backend, and AI service to that directory. The public frontend connects to the backend through HTTPS. The backend health check confirms that the API, database, and AI service are ready for the demo.

This design is practical for a final university project because it can be removed later as one temporary project without touching the existing unrelated server application.

4.3 DATABASE DESIGN

4.3.1 Database Overview

The final database is centered on the user. Every course, task, available slot, focus session, schedule, and behavioral record belongs to a specific user. This design prevents one user's academic data from mixing with another user's data.

4.3.2 Narrative Data Description

A user creates courses. A course can have many tasks and many focus sessions. A task belongs to one course and can become done after the student completes it. A focus session may connect to a task or only to a course, which supports the case where the student studies generally in a course without a selected task.

4.3.3 Entity Relationship Diagram (ERD)

User	owns courses, tasks, slots, focus sessions, and behavior records
Subject/Course	name, difficulty, priority, optional midterm date, optional final date
StudyTask	title, estimated minutes, actual minutes, deadline, priority, status
FocusSession	start/end time, course, optional task, mode, focus rating
AvailableSlot	day, start time, end time, saved study availability
AiSchedule	request payload, response payload, selected date, slot statuses
BehavioralLog	fatigue, focus ratings, snooze data, burnout support signals

4.3.4 Data Integrity and Constraints

The backend validates course ownership before creating tasks or focus sessions. Duplicate course names are checked per user. Passwords are hashed and are never stored as plain text. Course exam dates are nullable, so an empty midterm or final field is valid and does not create a false exam in the calendar.

4.4 HUMAN-MACHINE INTERFACE

The human-machine interface was improved to reduce confusion. The final screens use a direct student path: sign in, see today, open the calendar, manage courses, start focus, then review Study Pulse. The screenshots below use a demo account with a realistic team member email.

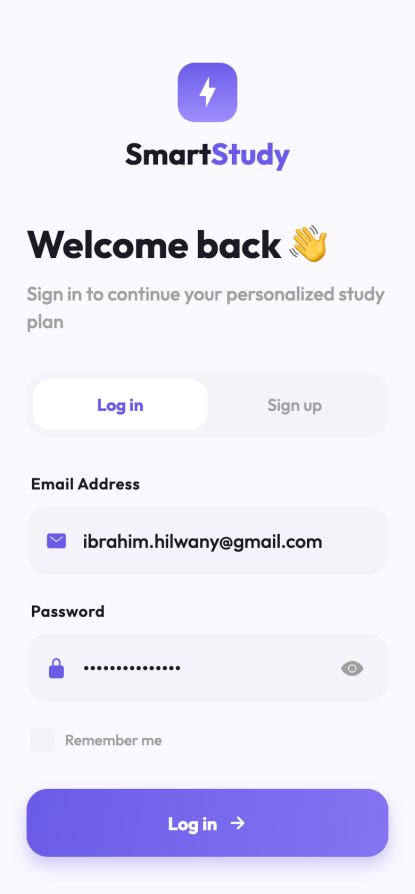


Figure 1. Email login for the demo account.

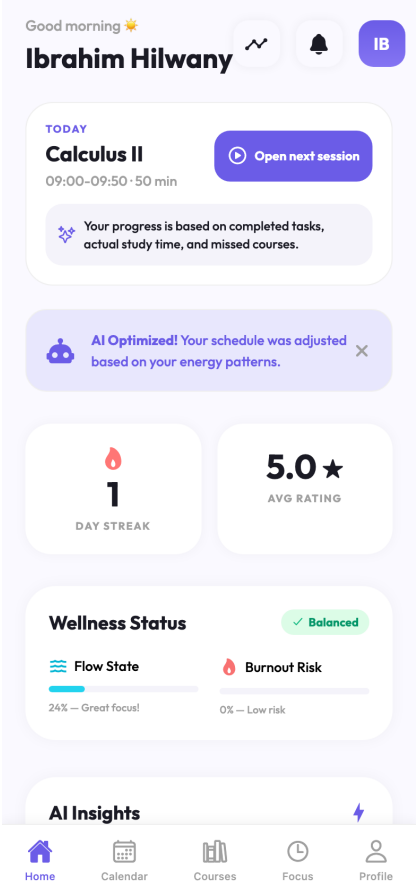


Figure 2. Home dashboard and today progress.

4.4.1 Functional Usage Description

A student can open the calendar, select a day, create tasks for that day, and later mark them as done. Completed tasks remain visible so the student can see progress in the same place where new work is planned.

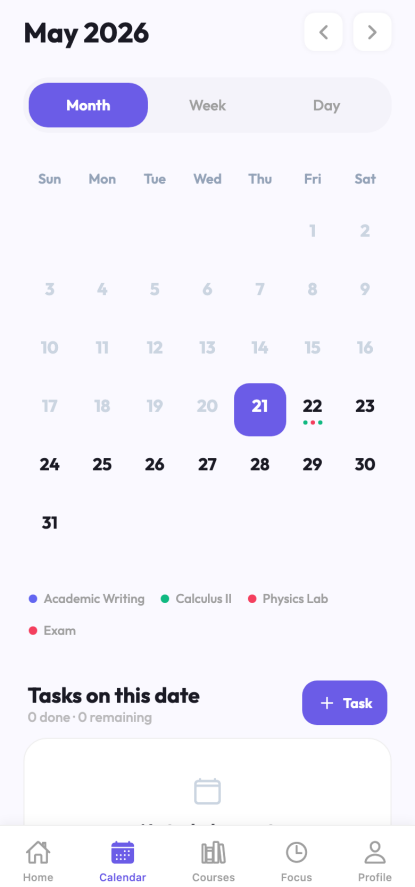


Figure 3. Calendar day with tasks.

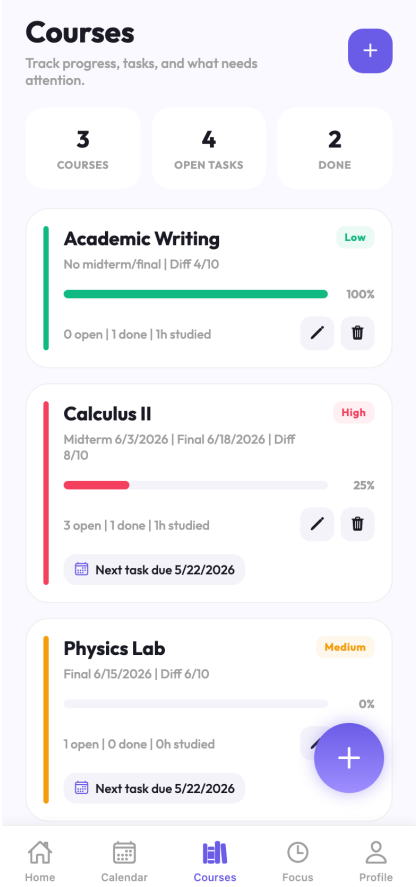


Figure 4. Courses overview with progress.

4.4.2 Inputs

The primary inputs are email, password, course name, priority, difficulty, optional midterm date, optional final date, task title, deadline, estimate, available study slots, and focus rating after a session.

4.4.3 Outputs

The main outputs are the daily plan, task list, course progress, exam reminders inside the calendar, completed state, focus session feedback, and Study Pulse analytics. These outputs were chosen to keep the student aware without turning the screen into a complicated analytics dashboard.

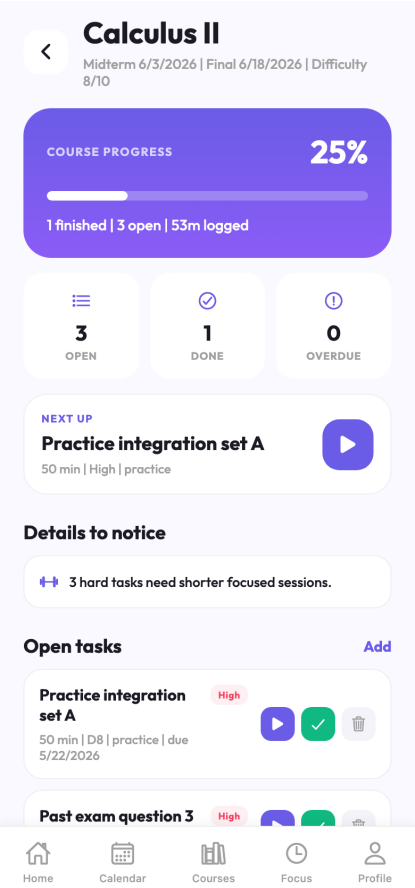


Figure 5. Calculus course detail.

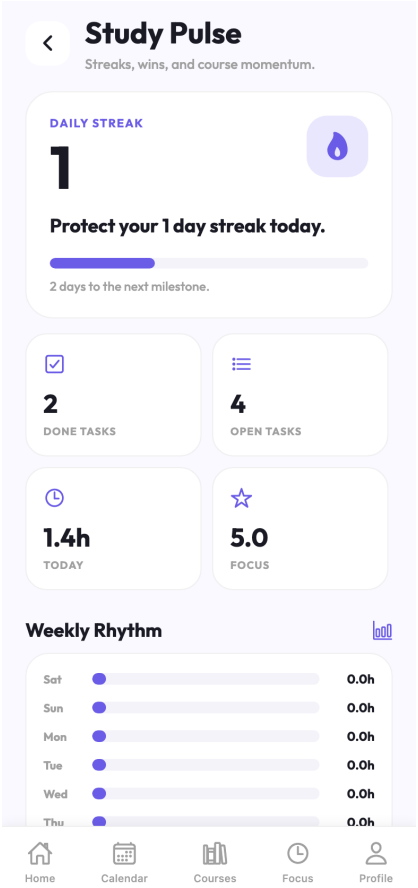


Figure 6. Study Pulse analytics.

4.5 DETAILED DESIGN

4.5.1 Software Detailed Design

The final detailed design follows service boundaries. The mobile screen calls an API service. The API controller validates the authenticated user and delegates the work to an application service. The service updates Entity Framework models and returns DTOs. The frontend maps those DTOs into screen cards and actions.

Course flow	Subjects service validates name, difficulty, priority, optional midterm/final, then returns Course UI data.
Task flow	Task service validates subject ownership and saves estimate, deadline, priority, and status.
Focus flow	Focus context starts and completes sessions, then analytics uses saved duration and rating.
Schedule flow	Schedule service builds data and chooses rule-based or AI path based on useAi.

Without AI, the schedule can rank work by available slots, deadlines, course priority, task priority, and difficulty. This answers the question of how the system decides what is easier or harder when AI is not used: the student supplies difficulty, and the system uses actual completion history and task estimates as supporting signals.

With AI, the backend sends the structured request to the FastAPI engine. The engine calculates analysis values, then Gemini can produce a more personalized plan. If that process is not available, the rule-based plan is still enough for the student to continue studying.

5. IMPLEMENTATION

5.1 Overview of the AI Engine Implementation

The AI engine is implemented as a Python FastAPI microservice. Its main endpoint receives courses, tasks, available slots, and behavioral values. It returns analysis results and a schedule structure that the backend can save and the mobile app can show.

5.2 Core Components and Logic

5.2.1 Server Initialization and Routing

The backend initializes ASP.NET Core services, database context, Identity password hashing, JWT bearer authentication, Swagger configuration, health checks, and controllers. The final deployment starts the backend as its own service inside the temporary project directory.

5.2.2 Intelligence Layer (Gemini Integration)

The intelligence layer calculates personalized planning signals. During cold start, it uses heuristics because a new student does not yet have enough historical data. After more completed tasks and focus sessions exist, the service can use personalized machine learning values and Gemini Flash for schedule improvement. The LLM is asked for structured schedule output, not free conversation.

5.3 Functional Proof and End-to-End Testing

A final demo user was created with email/password authentication, then several courses, tasks, completed items, focus sessions, and analytics values were generated through the real backend API. The same account was opened in the mobile/web preview and screenshots were captured from the running application.

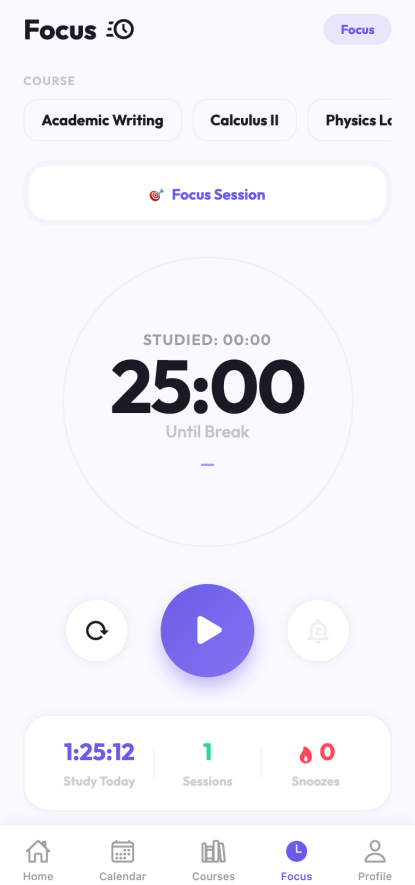


Figure 7. Focus timer connected to study work.

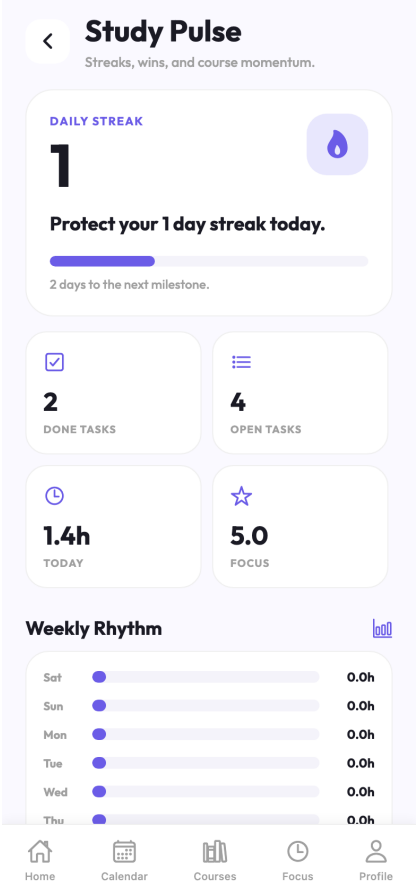


Figure 8. Analytics after demo activity.

5.4 Implementation Robustness

The final implementation was checked through API health checks, frontend screenshots, seeded demo data, and GitHub Actions deployment. The backend health endpoint reported the database and AI service as healthy after deployment. This confirms that the final demo is not only a local screenshot but a connected system.

The implementation also includes practical error handling. Duplicate course names are rejected by the backend, password rules are enforced, tasks must belong to the logged-in user, and the schedule service returns a fallback when AI optimization cannot complete. These checks reduce the chance of demo failure.

CI/CD was added so that pushing changes to the repository can update the temporary deployment. This makes the final project easier to maintain during the last presentation days because the team does not need to manually copy all services every time.

6. RESULT & CONCLUSION

6.1 Results and Discussion

The final result is a working Smart Study Planner application with frontend, backend, AI service, database, temporary HTTPS deployment, and CI/CD. The student can register, complete onboarding, add courses, add calendar tasks, complete tasks, start focus sessions, and see Study Pulse analytics.

6.1.1 Testing and Validation

Testing was done through backend API calls, demo account creation, frontend screenshots, health checks, and deployment verification. The demo data included three courses, several tasks, two completed tasks, a focus session, and analytics values such as one day streak, study hours today, focus score, done tasks, and open tasks.

6.1.2 What was Learned

The most important lesson is that a planner becomes useful only when the workflow is simple. A very smart system can still fail if adding tasks feels confusing. For that reason, the final design moved more power to calendar and course detail screens, and kept Study Pulse creative but not complicated.

6.2 Conclusion

The Smart Study Planner reached the main final goal. It is not only a design idea anymore, but a running application connected to real APIs and a hosted backend. It supports the student's real study routine: courses, optional exams, tasks, daily plans, focus time, and progress feedback.

6.3 Future Work and Improvements

- Add more flexible recurring tasks for students who study the same course every week.
- Improve notification timing using the student's real focus history.
- Add richer course reports while keeping the analytics simple on the main screen.
- Support exporting a semester summary for the student at the end of term.
- Improve accessibility and emulator/mobile touch accuracy testing across more devices.

The final project is complete for the current course requirement, and the remaining work is mostly product polish and deeper personalization. The base architecture is ready for that because the final system already keeps the important academic events and study behavior in structured data.

REFERENCES

[1] D. Kahneman and A. Tversky, Intuitive prediction: biases and corrective procedures, TIMS Studies in Management Science, 1979.

[2] B. J. Zimmerman, Becoming a self-regulated learner: an overview, Theory Into Practice, 2002.

[3] B. K. Britton and A. Tesser, Effects of time-management practices on college grades, Journal of Educational Psychology, 1991.

[4] C. Maslach, W. Schaufeli, and M. Leiter, Job burnout, Annual Review of Psychology, 2001.

[5] R. S. Baker and P. Inventado, Educational data mining and learning analytics, Learning Analytics, 2014.

[6] G. Siemens and R. Baker, Learning analytics and educational data mining, LAK Conference, 2012.

[7] A. K. D'Mello and S. K. Graesser, Dynamics of affective states during complex learning, Learning and Instruction, 2012.

[8] Scikit-learn developers, scikit-learn machine learning in Python documentation, accessed during project implementation.

[9] FastAPI documentation, building APIs with Python type hints and OpenAPI, accessed during project implementation.

[10] Microsoft, ASP.NET Core documentation for web APIs, dependency injection, middleware, and health checks.

[11] Microsoft, Entity Framework Core documentation for relational persistence and migrations.

[12] Microsoft, ASP.NET Core Identity password hashing and authentication guidance.

[13] IETF RFC 7519, JSON Web Token (JWT), 2015.

[14] React Native documentation, core components, state, and mobile interface patterns.

[15] Expo documentation, development server, mobile preview, and build workflow.

[16] SQLite documentation, relational database engine behavior and SQL constraints.

[17] Google AI documentation, Gemini API and structured generation guidance.

[18] GitHub Actions documentation, workflow syntax, secrets, runners, and deployment automation.

[19] OWASP Application Security Verification Standard, authentication and session management guidance.

[20] Nielsen Norman Group, usability heuristics for user interface design.

[21] WCAG 2.2, Web Content Accessibility Guidelines for readable and accessible interface design.

[22] Smart Study Planner source code, React Native frontend, final project repository.

[23] Smart Study Planner source code, ASP.NET Core backend, final project repository.

[24] Smart Study Planner source code, FastAPI AI engine, final project repository.

[25] Smart Study Planner API contract and integration notes, project documentation.

[26] Final demo account API seeding output, project testing artifact.

[27] Final frontend screenshots from the running Smart Study Planner application.

[28] Temporary deployment health-check output confirming backend, database, and AI service status.

[29] GitHub Actions deployment run for final project temporary hosting.

[30] Report_smart_study_planner.pdf, earlier project report artifact used for historical comparison.

[31] V1000000000000.docx.pdf, midterm report structure used as the base format for this final report.

[32] M. Fowler, Patterns of Enterprise Application Architecture, Addison-Wesley, 2002.

[33] E. Evans, Domain-Driven Design: Tackling Complexity in the Heart of Software, Addison-Wesley, 2003.

[34] R. C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design, Prentice Hall, 2017.

[35] Google Docs PDF renderer output from the original midterm document, used only for page structure reference.

[36] Team implementation notes and manual verification notes collected during the final project completion.